COS341 Project: SPL Compiler

User Manual of Group 31

1 Group members

Full name	Student number
Abdelrahman Ahmed	24898008
Morgan Calaca	24910882
Mutombo Kabau	24957102

2 Submission and requirements

The zip file `group-31.zip` holds the executable `group-31.jar` and this manual. We submit in the **1st Class** category with all phases from 2a to 3b. For every correct SPL program the compiler writes an executable C++ program. The signed declarations are on the last page.

- Java 8 or newer runs the jar on any operating system. The command `java -version` shows the installed version.
- The generated program is compiled with **g++**, which we prefer. gcc works as well with the option `-lstdc++` (Section 5).

3 Running the compiler

Put the test file `SPL.txt` into the folder of `group-31.jar`. Open a command prompt in that folder and run the jar. A double-click shows no console window and hides the messages.

```
java -jar group-31.jar
java -jar group-31.jar C:\tests\case3.txt
```

The second line reads another file. Paths with spaces need double quotes. Without an argument the jar looks for `SPL.txt` in the current folder and then in its own folder.

4 Output for a correct program

```
SPL.txt contains a correct SPL program.
The syntax tree has 61 nodes and was written to C:\COS341\tree.xml
The symbol table was written to C:\COS341\symbol-table.txt
The target code was written to C:\COS341\SPL.cpp
It is C++ code. Compile it with  g++ SPL.cpp -o SPL  and run SPL ...
```

All files go into the folder of the input file.

`tree.xml`	the syntax tree, which a web browser can display
`symbol-table.txt`	every declared name with its internal name and its type
`SPL.cpp`	the C++ program, named after the input (`case3.txt` gives `case3.cpp`)

5 Compiling and running the generated program

The code uses `#include <iostream>` and `std::cout`. Hence it is C++ code. We prefer g++:

```
g++ SPL.cpp -o SPL
SPL
```

The second line starts the program in `cmd` on Windows. Linux and macOS use `./SPL`. In PowerShell the command is `.\SPL`.

gcc 13 compiles the file as C++ too, but it does not link the C++ library on its own. It needs `-lstdc++`. Without that option, it reports undefined references to `std::cout`.

```
gcc SPL.cpp -lstdc++ -o SPL
```

Details for the run tests:

- No other options are needed. The first line of every generated file shows both commands.
- We tested g++ and gcc 13.3 on Linux as well as MinGW g++ 6.3 and MSYS2 g++ 14.1 on Windows. The code also passes `-pedantic-errors` from C++98 up to C++23.
- `-Wall` may warn about unused labels. The else label of an `if` is always created, as in the task sheet. The warning is harmless.
- Every `print` ends with a line break. Programs with `div` or a decimal point use `float`. All other programs use `int`.
- `and` / `or` are tested from left to right. The second condition is skipped when the first one decides the result.

6 Structure of the generated code

The code mine lies between `/* begin code mine */` and `/* end code mine */` and never runs. Each function has one comment there. It shows the parameters and local variables together with the translated body. C comments cannot be nested. Any comments inside the mined body are written as `(* *)`.

The executable part starts with `/* begin of executable C code */`. All variables are declared at the start of `main` with the value 0. Every call is replaced by the body of the function, and each copy gets its own labels and temporaries.

Conditions become jumps like `if ( t1 == t2 ) goto L5 ;`. The code has no `else` and none of `&&`, `||` or `!`. A `?` mark in a printed string is written as `\?`. Older compilers would otherwise read `??!` as a trigraph.

Name	Meaning
`v3_x`	variable or parameter `#x`
`f2_g`	function `#g`
`t7`	temporary value of a term
`p4`	argument of a call, computed before the parameters change
`f1_g_t9`	return variable of one inlined call of `f1_g`
`L12`	label

7 Error messages and exit codes

The compiler stops at the first phase that finds an error and writes no C++ file. Each report gives the line and column of the error. A marker points at the offending token and a hint as well. All naming errors or all type errors are listed together, at most 20. Output files of an earlier run are deleted. Otherwise, they could be mixed up with the new results.

Exit code	Result	Example cause
0	correct program	
1	lexical or syntax error	`-0.0` or a missing `;`
2	input file missing or not readable	
3	naming error (phase 2a)	a name without a declaration
4	type error (phase 2b)	a void function used as a term

```
Naming error in SPL.txt on line 5, column 9:
  The variable #m is not declared.

    5 | print ( #m ) ;
      |         ^^

Hint: Declare #m in the variable list before the first ':' of the main
      program.
```

A program with both `mod` and `div` gets the message: *A float-integer-conflict might perhaps be possible.*

8 Troubleshooting

'java' or 'g++' is not recognized as a command
Install Java 8 or newer, or a C++ compiler such as MinGW-w64 or MSYS2. Then open a new command prompt.

Error: Unable to access jarfile
Change to the folder of the jar or give the full path of the folder.

Could not find the input file
Windows may hide the extension. A file shown as `SPL.txt` could be `SPL.txt.txt`.

undefined reference to std::cout
gcc was used without `-lstdc++`. Add the option or use g++ instead.

9 Declarations

Declaration 1: submission category

We, the members of group 31, declare that we submit our project in the **1st Class** category. Our compiler includes every phase up to the generation of executable target code. All members of the group agree with this choice.

Full name	Student number	Signature	Date
Abdelrahman Ahmed	24898008		2026/10/25
Morgan Calaca	24910882		2026/10/25
Mutombo Kabau	24957102		2026/10/25

Declaration 2: contents of the ZIP file

We, the members of group 31, declare that every member of the group is fully aware of the whole contents of the ZIP file `group-31.zip`. Every member agrees with the submission of the ZIP file in this form.

Full name	Student number	Signature	Date
Abdelrahman Ahmed	24898008		2026/10/25
Morgan Calaca	24910882		2026/10/25
Mutombo Kabau	24957102		2026/10/25